

Tensor-compiler-based finite-element electromagnetics: cross-validating a Burn-native $H(\text{curl})$ solver against Palace on a superconducting transmon

Robb Walters*

Crutcher Dunnivant†

July 14, 2026

Abstract

The machine-learning ecosystem has produced general-purpose tensor-compiler stacks — batched kernels, JIT compilation, transparent multi-backend portability — representing one of the largest sustained engineering investments in numerical computing. We ask whether that investment can be inherited by classical mesh-based electromagnetics, and answer with a measured, honestly bounded yes. GEODE-FEM is a full-wave 3D $H(\text{curl})$ finite-element solver built in Rust on the Burn tensor framework: element assembly is batched $[n_{\text{elem}}, 6, 6]$ tensor algebra, the matrix-free operator is a gather $\rightarrow$ batched-matmul $\rightarrow$ scatter-add pipeline, and the driven Krylov iteration runs on-device with an $O(1)$ -scalar per-iteration synchronization budget, retargeting CPU and CUDA from one codebase. The claim is held to a cross-validation evidence standard: on a real transmon-plus-readout-resonator geometry meshed once and consumed identically by both solvers, GEODE-FEM reproduces Palace — the MFEM-based reference solver for superconducting-qubit design — to within 0.033% across all six eigenmodes (worst case 0.032%; junction LC mode 0.001%), with exact analytic tripwires guarding against coincidental agreement. Performance is reported as measured, wins and losses alike: on a matched same-session head-to-head at the physical target (6 modes, 4.5 GHz, 133k DOFs), geode-fem’s serial direct-factorization eigensolve on a single core (28.7 s) beats Palace on eight MPI ranks (44.5 s) in absolute wall clock while consuming $\sim 12\times$ fewer core-seconds — a per-core-efficiency, direct-versus-iterative result, not a parallelization claim (8-thread geode gives essentially no speedup here). The direct shift-invert is target-insensitive while Palace’s iterative Krylov+AMS degrades off-target, so the gap *widens* at 20 GHz ($6.7\times$ serial); the advantage is scale-bounded, inverting at $\sim 1.16\text{M}$ DOFs where the direct factorization is OOM-killed and Palace completes. Meanwhile the GPU driven-solve scaling cell is an honest negative — at $\leq 25,695$ edges the CUDA-f32 matrix-free path loses to every CPU configuration (assembled-CSR COCG is $44\times$ faster at the top size), reaching parity only against its own algorithm on CPU at $\sim 26\text{k}$ edges, with f32 accuracy flooring near 10^{-3} . Honest physics is the spine throughout: the model contains no $\sim 4\text{GHz}$ qubit mode *by construction*, and a disclosed spurious mode is diagnosed through a three-step gauge/projection arc — tree-cotree elimination (spectrum-shifting, 1.64%), bulk divergence-free projection (cavity-preserving but junction-deflating), and a port-aware rank-1 re-admission that lands all six modes at $\leq 0.029\%$ — and characterized as a port-surface artifact. Every number traces to a committed artifact.

1 Introduction

The tensor-compiler stacks built for machine learning — PyTorch, JAX, and, in Rust, Burn ([Tracel AI, 2024](#)) — embody an enormous, continuously maintained engineering investment: JIT kernel

*These authors contributed equally.

†These authors contributed equally.

generation, batched dense linear algebra, memory planners, and single-source portability across CPU, CUDA, and WebGPU backends. That infrastructure was built for neural networks, but nothing in it is specific to them. This paper asks a systems question: *can a production-grade computational electromagnetics solver be built on a general-purpose ML tensor-compiler stack* — inheriting its kernels, its JIT, and its backend portability — *rather than on the domain-specific FEM infrastructure the field has built for itself*? And it insists on an unforgiving evidence standard for the answer: same-mesh, same-physics cross-validation against the reference solver, at production accuracy, with every number traced to a committed artifact.

The mapping is natural once stated. Finite-element assembly is embarrassingly element-parallel dense algebra: for first-order Nédélec elements, the curl-curl and mass contributions are per-tetrahedron 6×6 matrices, which a tensor framework expresses as one batched $[n_{\text{elem}}, 6, 6]$ kernel. Matrix-free operator application is a gather (global edge DOFs to element-local vectors), a batched matrix multiply, and a scatter-add — exactly the primitives ML frameworks optimize. A Krylov iteration is a short loop of such applies plus reductions, which can stay resident on an accelerator with a constant number of scalar host-device synchronizations per iteration. GEODE-FEM (“geode-fem”) realizes this mapping in Rust on Burn (Section 3), with honest constraints stated up front: the CUDA backend is f32-only today, and the sparse direct factorization at the heart of the eigensolve remains a classical CPU component outside the tensor-compiler story.

The validation vehicle is deliberately demanding. Palace (Palace developers, AWS Center for Quantum Computing, 2023), an MFEM-based (Anderson et al., 2021; Andrej et al., 2024) parallel solver from the AWS Center for Quantum Computing, is the open-source solver of record for superconducting-qubit electromagnetic design, typically driven by geometry from DeviceLayout.jl (AWS Center for Quantum Computing, 2025; Peairs and Carson, 2025). Mode frequencies and participation ratios from such simulations feed directly into device Hamiltonians (Koch et al., 2007; Mineev et al., 2021a; Nigg et al., 2012), so the accuracy bar is production-grade, not demonstration-grade. Notably, this influential stack has never been the subject of a peer-reviewed or preprint publication — Palace exists as a repository, a blog post (AWS Quantum Technologies Blog, 2023), and a workshop talk; the DeviceLayout.jl transmon workflow as a blog post and a conference talk (Peairs, 2025; Peairs and Carson, 2025) — and no independent, quantitative, third-party validation of it exists in the literature. This paper therefore does double duty: the cross-validation that anchors the architecture claim is also the first independent validation of the reference stack itself. The comparison is same-mesh, same-physics, same-junction-model — the geometry is meshed *once* and both solvers consume the identical mesh, first-order discretization, and lumped reactive-shunt junction — so agreement claims are about formulation and solver correctness, not meshing luck.

Our contributions are:

- **Evidence that a general-purpose ML tensor stack can carry production-accuracy computational electromagnetics.** To our knowledge, geode-fem is the first full-wave 3D $H(\text{curl})$ /Nédélec FEM solver built on a general-purpose ML tensor-compiler stack and validated at production accuracy against the reference solver: on the identical mesh at matched order it reproduces all six of Palace’s eigenmodes to $\leq 0.033\%$ (worst case 0.032%), with the junction LC mode agreeing to 0.001% (Section 7). The same result is the first independent validation of the Palace/DeviceLayout.jl transmon stack.
- **An architecture for FEM as a tensor-compiler workload.** Batched element-local assembly, a matrix-free gather→batched-matmul→scatter-add operator, and an on-device Krylov loop with an $O(1)$ -scalar synchronization budget, retargeting CPU/CUDA from one codebase — with the current boundaries (f32-only CUDA; factorization-bound eigensolve) stated as constraints, not footnotes (Section 3).

- **Honest performance accounting on both axes.** On CPU, a matched same-session comparison at the physical target (6 modes, 4.5 GHz, 133k DOFs) shows geode-fem’s serial direct-factorization eigensolve on one core (28.7s) beating Palace on eight ranks (44.5s) in absolute wall clock at $\sim 12\times$ fewer core-seconds; the direct shift-invert is target-insensitive where Palace’s iterative solve degrades off-target, and the advantage is scale-bounded — it inverts at $\sim 1.16\text{M}$ DOFs where the direct factorization is memory-bound (Section 10). On GPU, a same-host four-configuration scaling study is reported as the honest negative it is: the CUDA-f32 matrix-free driven solve loses to every CPU configuration at every measured size, with a single directional positive — monotonic convergence to parity against the same algorithm on CPU at $\sim 26\text{k}$ edges (Section 11).
- **A reproducible same-mesh cross-solver methodology.** Oracle-first benchmarking — exact analytic tripwires, inverse tests that must fail, participation-based mode identification, and a fully scripted fixture-to-comparison pipeline (Sections 6 and 12).
- **Honest-negative reporting as a first-class result.** The model’s lack of a $\sim 4\text{GHz}$ qubit mode is explained and confirmed by both solvers (Section 8); a spurious mode leaked by geode-fem’s un-projected Lanczos is disclosed, *diagnosed through a measured three-step gauge/projection arc*, characterized as a port-surface artifact, and resolved by a port-aware projection that retains all six physical modes (Section 9).

Section 2 positions this work along both axes — FEM-on-tensor-stacks and qubit-EM validation; Section 3 presents the architecture; Sections 4–5 define the benchmark problem and the solver contrast; the remaining sections present methodology, results, and reproducibility details.

2 Related Work

FEM by code generation — and the libCEED foil. Generating high-performance FEM kernels from high-level descriptions is a two-decade tradition: the FFC form compiler (Kirby and Logg, 2006), the UFL weak-form language (Alnæs et al., 2014), Firedrake’s composition-of-abstractions approach (Rathgeber et al., 2017), and the structure-preserving TSFC compiler (Homolya et al., 2018) all compile variational forms to optimized element kernels. The modern GPU expression of this tradition is libCEED (Brown et al., 2021) — matrix-free, element-based operator evaluation — together with MFEM’s partial-assembly GPU path (Andrej et al., 2024). The positioning matters for this paper: *Palace itself runs on that stack* — MFEM with libCEED-class backends, a domain-specific element-kernel JIT built by and for the FEM community. geode-fem’s bet is categorically different: outsource kernel generation, scheduling, and backend portability to a *general-purpose ML tensor stack* whose engineering is amortized across the entire ML ecosystem, and phrase assembly and matrix-free application as batched tensor algebra. Form compilers generate FEM kernels; the tensor-compiler approach reuses someone else’s compiler and makes FEM look like its native workload. The comparison in this paper is therefore not “hand-written versus generated” but “domain-specific generator versus general-purpose stack” — with the same physics on both sides.

General-purpose tensor compilers and FEM on ML frameworks. The compiler lineage geode-fem’s substrate inherits runs through Halide’s algorithm/schedule decoupling (Ragan-Kelley et al., 2013) and TVM’s end-to-end tensor-program optimization (Chen et al., 2018); we found no peer-reviewed EM or FEM solver built directly on that lineage — its scientific-computing uptake has come via the ML frameworks. There, JAX-FEM (Xue et al., 2023) is the canonical

example (nodal-Lagrange continuum mechanics on JAX, GPU via XLA, differentiable), with PyTorch-FEA (Liang et al., 2023) the biomechanics analog. Closest to this work is the concurrent TensorMesh/TensorGalerkin framework (Wen et al., 2026) with its companion sparse-linear-algebra library torch-sla (Chi and Wen, 2026): general differentiable Galerkin assembly as batched tensor operations on PyTorch, whose demonstration suite includes magnetostatics via a stabilized *nodal* curl-curl formulation. Its existence independently supports the architectural thesis — FEM assembly as batched tensor algebra on an ML framework is now a genus, not a one-off — while sharpening what remains distinct here: TensorMesh is nodal rather than $H(\text{curl})$ -conforming, its electromagnetic examples are magnetostatic rather than full-wave generalized eigenproblems with lumped-element loading, and it reports no cross-validation against a reference EM solver at production accuracy. To our knowledge, no *full-wave 3D $H(\text{curl})$ /Nédélec* FEM solver has been built on a general-purpose ML tensor stack *and validated at production accuracy against the reference solver* before this work.

Differentiable and GPU electromagnetics. The differentiable-EM literature reaches Maxwell through structured grids: autograd FDFD/FDTD in ceviche (Hughes et al., 2019), JAX-based inverse design under foundry constraints (Schubert et al., 2022), the Meep adjoint pipeline (autograd/JAX around a classical FDTD core) (Hammond et al., 2022), the JAX-native FDTD framework (Mahlau et al., 2026), and differentiable RCWA in Meent (Kim et al., 2024). These codes get Maxwell onto ML frameworks by accepting rectilinear grids and low-order stencils; the transmon problem requires body-fitted tetrahedra, anisotropic sapphire, and $H(\text{curl})$ conformity — exactly the regime that literature has not reached. Adjacent but distinct genera: purpose-built differentiable kernel DSLs (DiffTaichi (Hu et al., 2019); Φ_{Flow} (Holl et al., 2020)), where the language is built *for* simulation rather than reused from ML; neural-ansatz Maxwell eigensolvers such as FieldTNN (Jiang et al., 2024), where a network *replaces* the discretization rather than executing it; and the hand-written-CUDA GPU FEM tradition for electromagnetics (Meng et al., 2014), which targets the same hardware at the cost of bespoke kernel engineering — the specialization-for-portability trade the tensor-compiler approach inverts.

Electromagnetic simulation workflows for superconducting qubits. The closest prior work on the validation axis consists of two recent Palace-based workflow papers. Sommers et al. (2025) present SQDMetal, an open workflow coupling Qiskit Metal, Gmsh, and Palace, and cross-check Palace against the commercial solvers COMSOL and Ansys HFSS on qubit and resonator geometries, reporting agreement at roughly the 0.02–0.13% level. Ye et al. (2025) build a Palace-centered layout-to-Hamiltonian workflow and benchmark it against cryogenic measurement, with resonator frequencies reproduced to within 0.3%. Both efforts treat Palace as a component to be *wrapped and trusted*: validation runs against commercial tools or experiment, through independently constructed models and meshes, so residuals mix solver error with meshing and model-entry differences. Neither is a same-mesh, same-formulation comparison against an independently *implemented* solver. Our study eliminates meshing and model entry as confounders — identical MSH fixture, identical lumped-shunt junction, first-order Nédélec elements on both sides — so residuals isolate formulation and solver correctness; to our knowledge it is the first independent third-party validation of Palace itself. The broader open ecosystem for qubit electromagnetic design includes Qiskit Metal and its quasi-lumped analysis methods (Minev et al., 2021b) and the SQuADDS validated design database (Shanto et al., 2024); the physics framing for extracting qubit parameters from field modes rests on the transmon (Koch et al., 2007), black-box quantization (Nigg et al., 2012), and energy-participation-ratio (Minev et al., 2021a) literature.

Verification, validation, and cross-solver benchmarking. Code-to-code comparison on precisely specified benchmark problems is an established genre in computational electromagnetics: the TEAM (Testing Electromagnetic Analysis Methods) workshop tradition has run such comparisons since 1986.¹ In the V&V vocabulary of Roache (1997) and Oberkampf and Roy (2010), the present study is verification-side evidence: our analytic tripwires are code-verification instruments in exactly Roache’s sense, and the same-mesh protocol is a code-to-code comparison in Oberkampf and Roy’s taxonomy. Multi-solver comparisons on shared electromagnetic problems have precedent in nano-optics, where Hoffmann et al. (2009) compared FEM, FDTD, and FIT codes on plasmonic antennas. We import that discipline to the superconducting-qubit domain and add two elements the genre often lacks: exact analytic tripwires that must move in known ways, and honest-negative reporting of what the model does *not* contain.

Open-source solver ecosystems and numerics. Mature open electromagnetics tools are normally accompanied by a citable methods paper: GetDP (Dular et al., 1998), DOLFIN/FEniCS (Logg and Wells, 2010), Gmsh (Geuzaine and Remacle, 2009), and scikit-rf (Arsenovic et al., 2022) all have one; so does Palace’s substrate library MFEM (Anderson et al., 2021; Andrej et al., 2024). Palace and DeviceLayout.jl are outliers — the *library* is published, the solver product built on it is not — which is precisely the gap this benchmark narrows from the outside. On the numerics side, geode-fem’s eigenpath is a shift-invert Lanczos (Lehoucq et al., 1998) over a real symmetric pencil, factorized with the faer sparse LU (Kazdadi, 2026), with first-order Nédélec elements (Nédélec, 1980); Palace’s eigenpath is a SLEPc-class Krylov method (Hernandez et al., 2005) with divergence-free projection and auxiliary-space (AMS) preconditioning (Hiptmair and Xu, 2007; Kolev and Vassilevski, 2009). The gauge-fixing literature relevant to Section 9 is the tree-cotree tradition (Albanese and Rubinacci, 1988; Manges and Cendes, 1995).

3 GEODE-FEM: Finite Elements as a Tensor-Compiler Workload

geode-fem is written in Rust on the Burn tensor framework (Tracel AI, 2024), whose cubec1 JIT compiles tensor kernels for CPU, CUDA, and WebGPU backends from one source. The design principle is to phrase every throughput-critical FEM stage as the batched dense algebra the stack already optimizes, and to be explicit about which stages do not (yet) fit.

Batched element-local assembly. For first-order Nédélec tetrahedra the element curl-curl and mass contributions are 6×6 matrices depending on per-element geometry factors and material tensors (including the rotated anisotropic sapphire permittivity of Section 4.1). geode-fem computes them for all elements at once as batched $[n_{\text{elem}}, 6, 6]$ tensor operations — one kernel launch per assembly stage rather than an element loop — then either scatters into a global sparse matrix (the assembled path used by the f64 CPU eigensolve) or retains the batch for matrix-free application.

Matrix-free operator application. The matrix-free apply of $(K + i\omega\text{-terms})x$ is a three-stage tensor pipeline: *gather* global edge coefficients into element-local $[n_{\text{elem}}, 6]$ vectors, *batched matmul* against the stored $[n_{\text{elem}}, 6, 6]$ element operators, and *scatter-add* back to global DOFs. Each stage is a native tensor-framework primitive; the backend (ndarray on CPU, CUDA on GPU) is a type parameter, not a code path.

¹The TEAM founding documents are workshop reports and proceedings without resolvable DOIs; we describe the tradition in prose rather than cite an arbitrary secondary source.

On-device Krylov with an $O(1)$ -scalar sync budget. The driven (deterministic frequency-sweep) solve runs a conjugate orthogonal conjugate gradient (COCG) iteration entirely on the device: matrix-free applies, axpy updates, and inner-product reductions all execute where the data lives, and only a constant number of scalars per iteration (convergence and recurrence coefficients) cross the host–device boundary. This is the architectural bet that device-resident Krylov amortizes once problems are large enough to fill an accelerator — Section 11 measures, honestly, how far below that threshold the current benchmark sizes sit.

Honest constraints. Two boundaries of the tensor-compiler story are stated as constraints rather than buried. First, the CUDA backend (burn-cuda 0.21) is f32-only today — the underlying cubeccl runtime currently disables f64² — so every GPU result in this paper is mixed-precision-qualified and reported with explicit accuracy deltas against the f64 CPU reference. Second, the eigensolve that produces the headline comparison is factorization-bound: its inner solves are faer sparse-LU factorizations (Kazdadi, 2026), a classical serial CPU component. The tensor-compiler story currently covers assembly and driven solves, *not* sparse direct factorization; the eigensolver gauge/projection work of Section 9 (repository issues #502/#503 and follow-ons) is a prerequisite for a future projected, iterative — and therefore GPU-viable — eigenpath.

4 Benchmark Problem: Geometry, Mesh, and Junction Model

4.1 Geometry and mesh provenance

The device is the `SingleTransmon` example shipped with `DeviceLayout.jl` v1.15.0 (AWS Center for Quantum Computing, 2025): a transmon with readout resonator on a sapphire substrate, enclosed in a vacuum box. The sapphire is anisotropic, with permittivity tensor $\varepsilon = R \text{diag}(9.3, 9.3, 11.5) R^\top$ rotated approximately 36.87° in-plane. The `DeviceLayout.jl` workflow emits seven named physical groups (metal, substrate, vacuum, junction lumped-element surface, two readout ports, exterior boundary) and drives Gmsh (Geuzaine and Remacle, 2009) to produce a tetrahedral mesh, exported as an MSH 4.1 fixture and pinned by SHA-256 (Section 12).

The mesh has 22,684 nodes and 133,314 tetrahedra, yielding 156,863 Nédélec edge unknowns, of which 133,108 interior degrees of freedom remain after eliminating perfect-electric-conductor (PEC) boundary edges. PEC is applied on all metal surfaces and the exterior box; the two readout ports are left open (lossless) in this first version, so no resistive element enters and the eigenproblem pencil stays real symmetric. Figure 1 shows the geometry with physical groups color-coded.

4.2 The junction model: a lumped reactive shunt

The Josephson junction is modeled identically in both solvers as a lumped reactive shunt — a parallel $L \parallel C$ element with $L = 14.860 \text{ nH}$ and $C = 5.5 \text{ fF}$ — attached to the four-triangle `lumped_element` surface group Γ . Following the uniform lumped-port construction, with port length ℓ and width w , the shunt contributes

$$K_{\text{port}} = \frac{\ell}{w L} S_\Gamma \quad \text{and} \quad M_{\text{port}} = \frac{C \ell}{w} S_\Gamma, \quad (1)$$

where S_Γ is the port-surface matrix assembled on Γ . The inductive term is frequency-independent and adds to the stiffness matrix; the capacitive term adds to the mass matrix. The eigenproblem is then the real symmetric generalized pencil

$$(K + K_{\text{port}}) x = \omega^2 (M + M_{\text{port}}) x. \quad (2)$$

²Upstream constraint of the burn/cubeccl stack at version 0.21 (Tracel AI, 2024); tracking-issue URL to be added at revision. TODO(operator).

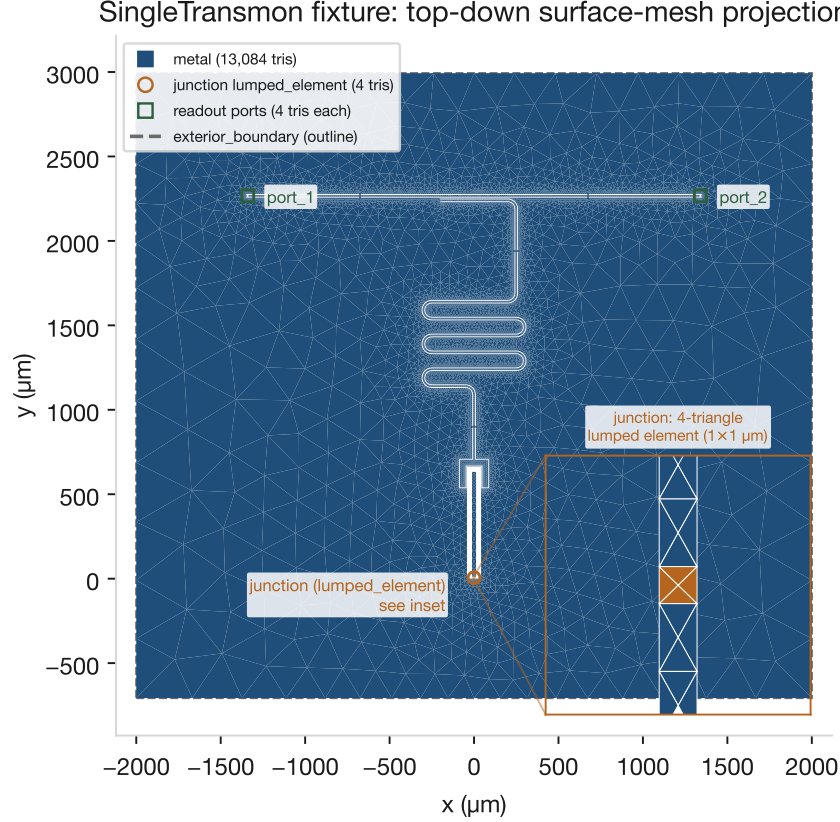


Figure 1: Benchmark geometry: top-down projection of the committed surface mesh (the SHA-256-pinned MSH 4.1 fixture) for the DeviceLayout.jl v1.15.0 `SingleTransmon` transmon and readout resonator. Surface physical groups are color-coded — the metal ground plane (with the coplanar-waveguide feedline, meander resonator, and transmon slot as etched gaps), the four-triangle junction `lumped_element` surface (enlarged in the inset, actual triangulation shown), the two readout ports, and the exterior-boundary outline; the `substrate` and `vacuum` volume groups fill the enclosing box. Both solvers consume this identical mesh (22,684 nodes; 133,314 tetrahedra).

In Palace, the same physics is entered as a reactive `LumpedPort` ($L \parallel C$, no resistance) on the same surface group; the readout ports are configured open (no $50\ \Omega$ termination), so Palace’s pencil is likewise real. This one-to-one correspondence of the junction treatment is what makes the comparison same-physics as well as same-mesh.

5 Solver Architectures Under Comparison

The two solvers share the mathematical problem (Eq. 2) and nothing else: no code, no language, no linear algebra, no eigensolver strategy — and, per Section 2, they sit on opposite sides of the domain-specific-versus-general-purpose compiler divide. Table 1 summarizes the contrast.

geode-fem assembles the full-tensor Nédélec system in double precision via the batched element kernels of Section 3, reduces to interior DOFs, and solves the pencil with a real shift-invert Lanczos iteration whose inner solves are direct sparse-LU factorizations. It runs in a single process with no intra-solve parallelism today. Palace assembles via MFEM, applies a divergence-free projection to confine the Krylov iteration to the physical subspace, preconditions with auxiliary-space methods

Table 1: Solver architectures compared. The two implementations share the discretized problem definition (identical mesh, first-order Nédélec elements, identical lumped-shunt junction) and are otherwise independent.

	geode-fem	Palace
Language / substrate	Rust / Burn tensors (f64)	C++ / MFEM
Kernel provenance	general-purpose ML tensor stack	domain-specific FEM stack (libCEED-class)
Assembly	batched-tensor Nédélec (sparse global)	MFEM $H(\text{curl})$
Eigensolver	shift-invert Lanczos	SLEPc-class Krylov
Inner linear solve	faer sparse LU (direct)	AMS-preconditioned iterative
Spurious-mode control	none on headline path (§9)	divergence-free projection
Parallelism	single process	MPI-distributed

(AMS) (Hiptmair and Xu, 2007; Kolev and Vassilevski, 2009), and distributes over MPI ranks. These choices have consequences for both correctness behavior (Section 9) and performance (Section 10); the point of the benchmark is to expose those consequences honestly rather than to declare a winner.

6 Validation Methodology

The benchmark protocol is oracle-first: every claim is anchored either to an exact analytic quantity or to a committed reference artifact, and the test suite is designed so that silent pipeline failures are caught by construction.

Same-mesh, same-physics protocol. The mesh is generated once, committed, and pinned by SHA-256. Both solvers consume it at matched (first) order with the identical junction model (Section 4.2). This removes meshing and model entry as confounders: any residual disagreement is attributable to formulation or solver implementation.

Analytic tripwires. Two exact quantities guard against coincidental or self-consistent-but-wrong agreement. First, the isolated junction LC resonance $f_{LC} = 1/(2\pi\sqrt{LC}) = 17.60$ GHz anchors the expected location of the junction mode. Second, a Josephson inductance-scaling tripwire: doubling L must move the junction mode by exactly $1/\sqrt{2}$. In our runs the junction mode moves from 17.4901 to 12.37 GHz, a ratio of 0.7071 — $1/\sqrt{2}$ to the reported precision (Figure 3); the measured tripwire values are committed alongside the benchmark results (Section 12).

Inverse tests that must fail. Alongside tests asserting agreement, the suite contains perturbed configurations (such as the doubled- L case) that are *required to disagree* with the committed oracle. A pipeline that still “passes” under such a perturbation is not measuring anything; the inverse tests make that failure mode loud.

Participation-based mode identification. Modes are identified, not eyeballed — but the identification must respect the fact that the two solvers expose *different* participation diagnostics. Within geode-fem, for each eigenvector x the stiffness-participation ratio

$$p = \frac{x^\top K_{\text{port}} x}{x^\top (K + K_{\text{port}}) x} \quad (3)$$

measures the fraction of pencil energy stored in the junction inductor and cleanly separates the junction LC mode ($p = 1.000$) from the resonator and box modes ($p = 0.000$ at the artifact’s three-decimal precision). Palace reports a per-port energy-participation (port-EPR) output, which

Table 2: Eigenmode agreement on the identical mesh (22,684 nodes / 133,314 tets / 156,863 Nédélec DOFs, 133,108 interior after PEC), first order in both solvers, junction as a lumped reactive shunt ($L = 14.860$ nH, $C = 5.5$ fF). geode-fem at commit 3174015; Palace at commit fba6a5b. All values are quoted verbatim from the committed `benchmarks/transmon_eigen/results.toml` (mode keys in the first column): geode-fem frequencies at the committed three decimals, Palace at full precision, and the committed per-mode `rel_err_pct`, which is computed against the rounded geode-fem values (precision disclosure in Section 12).

Mode		geode-fem (GHz)	Palace (GHz)	Δ (%)
<code>resonator</code>	readout resonator	5.153	5.151335830348	0.032
<code>mode_2</code>	box/cavity	15.465	15.46052107794	0.029
<code>junction_lc</code>	junction LC	17.490	17.49010903536	0.001
<code>mode_4</code>	box/cavity	18.693	18.69165792915	0.007
<code>mode_5</code>	box/cavity	20.703	20.69755679425	0.026
<code>mode_6</code>	box/cavity	26.088	26.08089940472	0.027

is a field-based, differently normalized quantity; on the committed run the two diagnostics do *not* rank the modes the same way (in Palace’s committed `port-EPR.csv` the 17.49 GHz junction mode carries the smallest-magnitude port-EPR of the six modes, all of which are $\leq 5 \times 10^{-4}$). The two quantities are complementary diagnostics, not a shared discriminant, and we do not compare them numerically. Cross-solver mode identification therefore rests on *frequency* agreement — the junction mode appears at 17.490 GHz (geode-fem) and 17.4901 GHz (Palace), matching the analytic anchor $f_{LC} = 17.60$ GHz — with geode-fem’s participation used to identify the junction mode on its own side and to expose the spurious mode of Section 9.

Determinism check. The Palace oracle is not a one-off: rerunning Palace from the committed configuration reproduced its committed `eig.csv` bit-for-bit.

7 Results: Cross-Solver Agreement

Table 2 presents the headline result: on the identical 133k-DOF mesh at matched order, all six modes agree to $\leq 0.033\%$ (worst case 0.032%), and the junction LC mode — the mode the junction model exists to produce — agrees to 0.001%. This is the evidence standard the architecture claim of Section 1 is held to: a solver whose assembly and operators are batched tensor algebra on an ML stack reproduces the domain-specific reference implementation at production accuracy on a real device geometry.

Three features of Table 2 deserve comment. First, the junction mode sits at 17.490 GHz (geode-fem) and 17.4901 GHz (Palace), 0.6% below the isolated-circuit anchor $f_{LC} = 17.60$ GHz — the small shift reflects the junction’s embedding in the field problem, and its near-coincidence with the analytic value in *both* solvers is the tripwire working as intended. Second, the mode identification is robust: within geode-fem, junction participation is $p = 1.000$ for the 17.49 GHz mode and $p = 0.000$ for every other physical mode, and the cross-solver assignment rests on frequency agreement against the analytic anchor rather than on comparing the two solvers’ differently normalized participation outputs (Section 6). Third, the agreement is between genuinely independent code paths — a batched-tensor assembly feeding a direct-factorization Lanczos in Rust, and a projected, preconditioned Krylov method in C++ — so a common implementation error is an unlikely explanation for the residual $\leq 0.033\%$ (worst case 0.032%), and the analytic tripwires exclude self-consistent-but-wrong agreement about the junction physics.

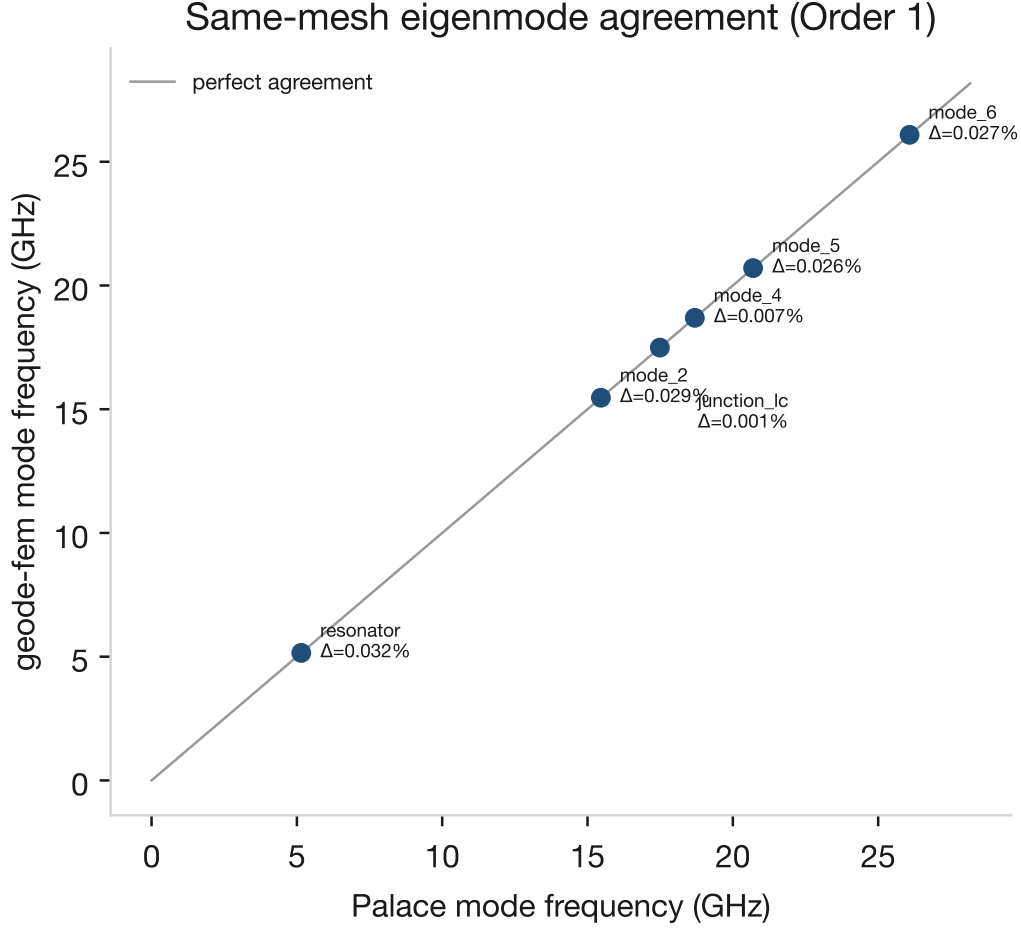


Figure 2: Mode-frequency agreement: geode-fem versus Palace on the identical mesh, with per-mode relative deviations annotated. Data from the committed benchmark artifacts (`benchmarks/transmon_eigen/results.toml` and Palace’s committed `eig.csv`).

8 The Absent Qubit Mode: An Expected Negative

A reader familiar with the DeviceLayout.jl blog workflow (Peairs and Carson, 2025) will notice that its reported spectrum spans $[4.14, 5.591]$ GHz — including a ~ 4 GHz qubit mode — while Table 2 contains no mode below 5.15 GHz. This is not a discrepancy to be minimized; it is a property of the model, and we state it plainly: **no ~ 4 GHz qubit mode exists in this junction model, by construction.**

The physical transmon frequency arises from the Josephson inductance resonating against the *pad (shunt) capacitance* of roughly 80–100 fF (Koch et al., 2007). The v1 lumped-shunt model of Section 4.2 gives the junction only its own 5.5 fF, so the junction’s resonance lands at 17.5 GHz, not near 4 GHz; the pad capacitance exists in the field solution but is not wired across the lumped element. Both solvers agree on the absence: Palace’s projected eigensolve, instructed to hunt at a shift of $\sigma = 4.5$ GHz, finds nothing below the 5.15 GHz resonator mode. The blog’s $[4.14, 5.591]$ GHz spectrum is therefore *not reproduced by this model*, and no agreement with it is claimed.

The route to genuine qubit quantities from this benchmark is energy-participation-ratio (EPR) post-processing over the field modes (Mineev et al., 2021a; Nigg et al., 2012), in which the junction’s

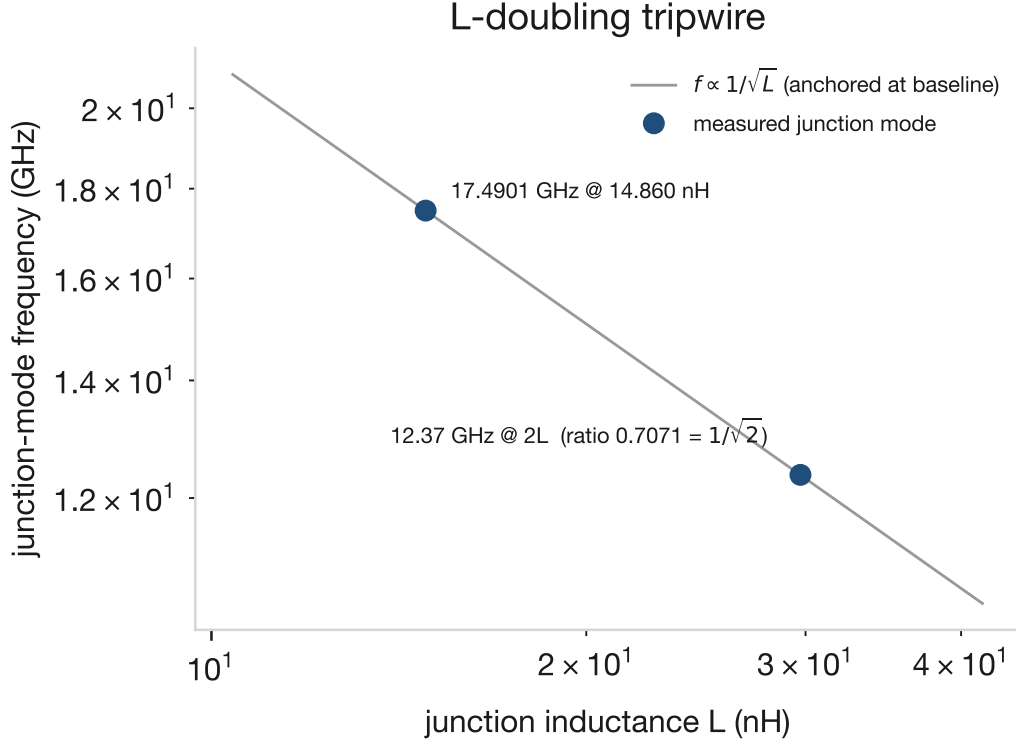


Figure 3: Josephson inductance-scaling tripwire: junction-mode frequency versus junction inductance L on log-log axes. Doubling L moves the junction mode from 17.4901 to 12.37 GHz — ratio $0.7071 = 1/\sqrt{2}$ — exactly as the $f \propto 1/\sqrt{L}$ scaling requires. Measured values from the committed `benchmarks/transmon_eigen/results.toml` tripwire block.

participation in each electromagnetic mode — exactly the quantity in Eq. 3 — feeds the quantization. A first quantum layer along that route has since been built and measured on this same mesh; Section 13 summarizes it in one forward-looking paragraph, and the full analysis is out of scope for the present cross-validation.

9 A Disclosed Spurious Mode: Diagnosed, Characterized, and Resolved

Honest cross-validation requires reporting where the solvers *disagree*, and there is exactly one such place. geode-fem’s headline Lanczos iteration operates on the raw pencil of Eq. 2 without a gauge projection. On this problem it leaks one spurious mode at 3.4528 GHz, below the physical band, localized on the junction surface with participation $p = 0.9942$. Palace, whose eigensolve projects onto the divergence-free subspace, shows no such mode — its spectrum is clean down through the $\sigma = 4.5$ GHz hunt of Section 8. The mode is excluded from Table 2 by documented criteria (absence from the projected Palace spectrum on the identical mesh; location far below the analytic junction anchor while carrying near-total junction participation). In an earlier version of this work that disclosure-plus- filtering was where the story ended; since then the artifact has been run to ground through a measured three-step gauge/projection arc, every step committed alongside the benchmark results (Section 12), and the mode is now diagnosed, characterized, and — at the solver level — resolved.

Step 1: tree-cotree elimination is not spectrum-preserving. The classical gauge for the curl-curl *source* problem is tree-cotree DOF elimination (Albanese and Rubinacci, 1988; Manges and Cendes, 1995): a spanning-tree decomposition of the edge graph identifies exactly $\text{rank}(d_{\text{interior}}^0) = 13,747$ gradient DOFs on this mesh (the count is proven bit-for-bit against the de Rham gradient rank), and eliminating them removes both the near-zero gradient cluster and the 3.4528 GHz mode. But for the *generalized eigenproblem* the physical eigenvectors have nonzero tree-edge components, so the constraint shifts the spectrum: the resonator moves from 5.1528 to 5.2372 GHz — a 1.64% shift, a converged eigenvalue stable across Krylov sizes, outside our $\leq 1\%$ bar. This honest negative is pinned as a regression test.

Step 2: bulk divergence-free projection preserves the cavity but deflates the junction.

The spectrum-preserving alternative is the M-orthogonal projector $P = I - G(G^T M G)^{-1} G^T M$ with $G = d_{\text{interior}}^0$, applied inside the Lanczos iteration — structurally the same divergence-free confinement Palace applies (Palace developers, AWS Center for Quantum Computing, 2023). Measured on the 133k-DOF fixture, it does what tree-cotree elimination could not: the cavity spectrum is preserved (resonator at 0.029% versus Palace, against the gauge’s 1.64% shift), every Ritz vector is divergence-free to $\sim 10^{-15}$, and the 13,747-mode near-zero gradient cluster collapses to a single survivor. But two measured facts show the *bulk* projection is incompatible with the lumped-inductor formulation. First, the spurious 3.4528 GHz mode *survives*: its eigenvector is genuinely solenoidal (divergence ratio $\approx 6 \times 10^{-15}$), so it is not a bulk-gradient artifact at all. Second, the *physical* junction LC mode is deflated away: measured directly on the raw ungauged junction eigenvector, its divergence ratio $\|G^T M x\|/\|x\|_M = 50.2$ is macroscopic and its projected norm $\|Px\|_M/\|x\|_M = 1.06 \times 10^{-4}$ is near zero — a lumped inductor is a quasi-static, curl-free flux path, so the junction mode lives almost entirely in the gradient subspace and the bulk projection cannot distinguish it from the artifacts it removes.

Step 3: port-aware rank-1 re-admission retains all six modes. The resolution re-admits exactly one direction: with $\hat{u} = (I - P) x_{\text{junction}}/\|(I - P) x_{\text{junction}}\|_M$ extracted from the ungauged junction eigenvector, the rank-1 M-orthogonal update $P' = P + \hat{u}\hat{u}^T M$ is a genuine projector onto (divergence-free subspace) $\oplus \text{span}\{\hat{u}\}$ — identity on the cavity modes *and* on the junction flux, annihilator of the other 13,746 gradient directions. The measured acceptance: all six physical modes agree with Palace to $\leq 0.029\%$ worst case, *including* the junction LC mode retained at 17.4901 GHz, with the gradient cluster gone. The headline comparison of Table 2 deliberately remains the committed ungauged benchmark (the port-aware solver is a composite — an ungauged junction extract feeding the projected band solve — and shipped after the headline measurement); its acceptance numbers are committed alongside.

What the spurious mode actually is. Two discriminating measurements characterize the 3.4528 GHz artifact. Under the same L-doubling harness as the physical tripwire, it moves to 2.4449 GHz — ratio 0.7081, essentially the junction $1/\sqrt{L}$ law ($1/\sqrt{2} = 0.7071$; a box or mesh mode would not move at all) — and 99.4% of its stiffness energy sits in the K_{port} surface term ($p = 0.9942$). It is therefore a second LC-like resonance of the S_Γ port-surface operator: a K_{port} -driven, junction-localized eigenmode with no cavity counterpart. Palace’s **LumpedPort** formulation represents port unknowns differently (eliminated, rather than added to the full-space stiffness as our S_Γ term), which is exactly why it has no Palace analogue — and, being solenoidal, no divergence-free projection, bulk or port-aware, removes it. On the headline path it remains filtered by the documented frequency-matching criteria; the difference is that the filter now excludes a

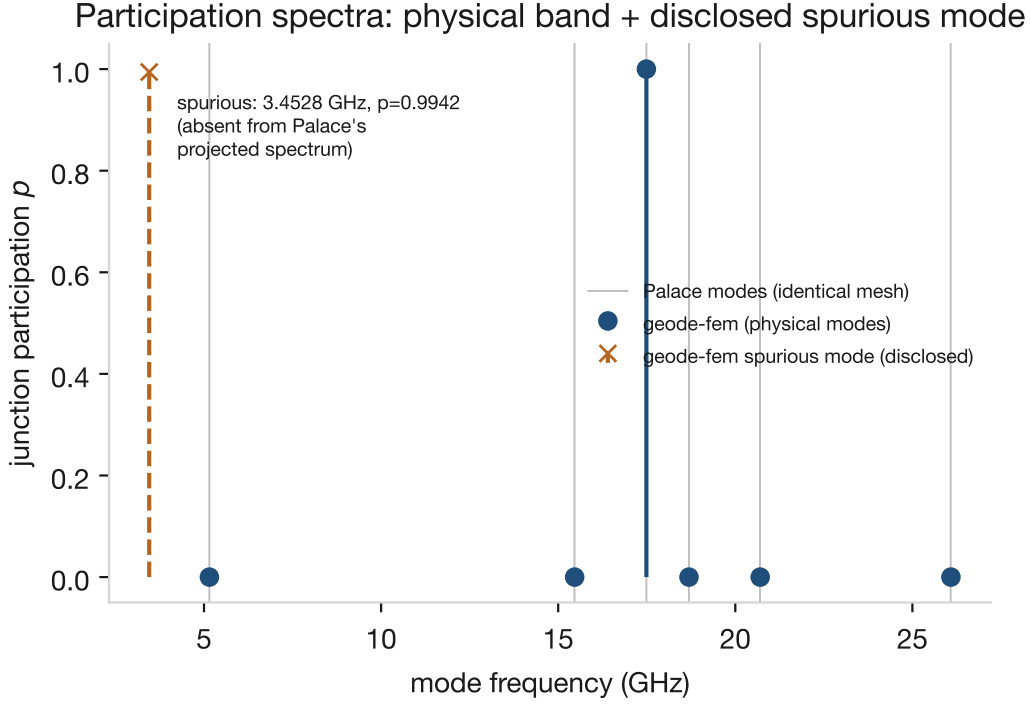


Figure 4: geode-fem junction-participation spectrum on the identical mesh, with Palace’s mode locations (from the committed `eig.csv`) overlaid. The physical junction LC mode appears at 17.49 GHz in both solvers’ spectra (geode-fem stiffness participation $p = 1.000$); geode-fem’s un-projected headline Lanczos additionally leaks one spurious junction-localized mode at 3.4528 GHz ($p = 0.9942$) with no Palace counterpart — characterized in Section 9 as a port-surface artifact obeying the junction $1/\sqrt{L}$ law. Palace’s field-based port-EPR is a differently normalized diagnostic and is not plotted on geode-fem’s participation axis (Section 6).

characterized artifact rather than an unexplained one. Figure 4 contrasts the two solvers’ spectra, spurious mode included. We regard the arc itself as a finding: it measures precisely what Palace’s divergence-free projection buys in a production eigensolver, and what a lumped-element formulation demands beyond it.

10 Performance: CPU Cell

10.1 Setup

All CPU measurements ran on a single Amazon EC2 `m6i.4xlarge` instance (8 physical cores / 16 vCPUs, ~61 GB usable RAM) in `us-west-2`.³ The cell is a *matched* head-to-head: on the same box in the same session, both solvers ran the identical mesh at the identical eigenproblem, each sampled at 1 and 8 cores/ranks (`GEODE_NUM.THREADS` $\in \{1, 8\}$ for geode-fem; `mpirun -np` $\in \{1, 8\}$ for Palace), at three workloads: the physical target (6 modes, 4.5 GHz), an off-target window (12 modes, 20 GHz), and a uniformly refined large-scale mesh (~1.16M interior DOFs). Each measurement covers the *full pipeline* — mesh load, assembly, solve, and output — timed with `/usr/bin/time`; timings are the mean over $n = 3$ runs. geode-fem ran at commit 3174015, Palace at commit fba6a5b, both release builds. The complete matched cell — per-run wall clocks, peak resident-set sizes, and instance/software provenance — is committed as `benchmarks/transmon_bench_cpu/results.toml`.

³On-demand cost approximately \$0.77/hour at the time of the runs; the entire CPU cell is reproducible for a few dollars.

Table 3: Matched CPU-cell wall-clock and memory, full pipeline (mesh load + assembly + solve + output), `m6i.4xlarge`, same box and session. geode-fem @3174015 uses a serial/threaded faer sparse-LU shift-invert Lanczos (direct); Palace @fba6a5b uses distributed MPI Krylov + AMS (iterative). Timings are the mean over $n = 3$ runs. Palace memory is the per-rank maximum single-process RSS, not an aggregate across ranks. At the physical target geode on *one* core (28.7 s) beats Palace on *eight* ranks (44.5 s) in absolute wall clock, and the gap widens off-target; at ~ 1.16 M DOFs the direct factorization is OOM-killed (63.9 GB peak on the ~ 61 GB box) while Palace completes. Values from the committed `benchmarks/transmon_bench_cpu/results.toml` (`[matched.*]` tables).

Workload	Solver (cores)	Wall (s)	Peak RSS
<i>Physical target: 6 modes @ 4.5 GHz, 133,108 interior DOFs</i>			
	geode-fem (1)	28.7	3.1 GB
	geode-fem (8)	29.0	3.1 GB
	Palace (1)	130.9	0.5 GB/rank
	Palace (8)	44.5	0.5 GB/rank
<i>Off-target: 12 modes @ 20 GHz, 133,108 interior DOFs</i>			
	geode-fem (1)	36.8	3.1 GB
	geode-fem (8)	26.6	3.1 GB
	Palace (1)	248.0	0.5 GB/rank
	Palace (8)	64.7	0.5 GB/rank
<i>Large scale: 6 modes @ 4.5 GHz, 1,157,564 interior DOFs</i>			
	geode-fem (1)	OOM-killed	63.9 GB
	Palace (8)	423.12	4.1 GB/rank

10.2 Results

Table 3 supports three readings that must be taken together; none of them is a “we parallelized better” claim.

Per-core efficiency and an absolute single-core win. At the physical target, geode-fem’s serial direct factorization on a *single* core (28.7 s) beats Palace on *eight* MPI ranks (44.5 s) in absolute wall clock, and does so while consuming $\sim 12\times$ fewer core-seconds (28.7 versus 356.0 core-s). This is a per-core-efficiency and direct-versus-iterative result, explicitly *not* a parallelization win: geode-fem’s own 8-thread run gives essentially no speedup over its 1-thread run at this target (29.0 versus 28.7 s), a limitation tracked as repository issue #518. The advantage comes from doing one sparse factorization at high per-core throughput rather than from spreading work across cores.

Target robustness. The direct shift-invert is *target-insensitive* — the cost of a single factorization at the shift is nearly independent of where the target sits — so moving from 6 modes at 4.5 GHz to 12 modes at the far-off-target 20 GHz barely changes geode-fem’s serial time (28.7 $\rightarrow$ 36.8 s). Palace’s iterative Krylov+AMS *degrades* off-target, where a poorly placed shift slows convergence (130.9 $\rightarrow$ 248.0 s at np1; 44.5 $\rightarrow$ 64.7 s at np8). Consequently the gap *widens* off-target — $6.7\times$ serial (248.0/36.8) and $2.4\times$ at 8-wide (64.7/26.6) — rather than being a fixed multiplier.

A memory-bound crossover. The direct factorization trades memory for this speed and robustness. It is faster on small-to-medium problems (28.7 s / 3.1 GB at 133k DOFs), but its fill-in grows super-linearly and hits a hard memory wall: on the uniformly refined ~ 1.16 M-DOF mesh it is OOM-killed (SIGKILL, exit 137, 63.9 GB peak RSS on the ~ 61 GB box, never completing), while Palace’s iterative solver completes at low, roughly rank-flat memory (423.12 s, 4.1 GB/rank). The

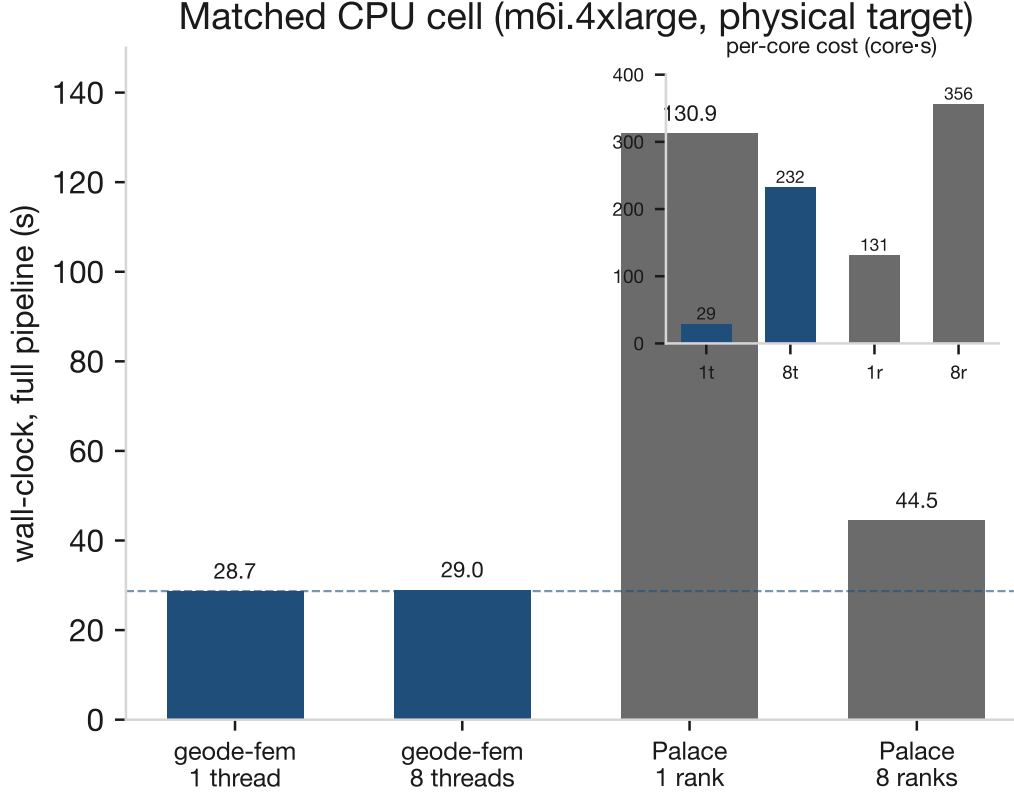


Figure 5: Matched CPU-cell wall-clock comparison at the physical target (6 modes, 4.5 GHz, 133k DOFs): geode-fem at 1 and 8 threads versus Palace at 1 and 8 MPI ranks. The inset reports per-core cost as core-seconds — geode on one core consumes 28.7 core-s against Palace’s 356.0 at 8 ranks ($\sim 12\times$). Full-pipeline timings from Table 3.

geode-fem advantage is therefore scale-bounded, and the crossover is memory-bound rather than a flop deficit. The scale path that removes the memory wall (not a bid to beat Palace’s large-scale wall time) is a matrix-free eigensolver (repository issue #524) with an AMS-style $H(\text{curl})$ preconditioner (#526) — an iterative inner solve that avoids the direct fill-in — with wall-clock-competitive scale tracked separately (#531); the f32 GPU route to that regime is reported as an honest negative below and in issue #520.

This is an architecture trade-off, not a ranking: geode-fem wins small-to-medium on speed, per-core efficiency, and target robustness; Palace wins at scale on memory, completing where the direct path OOMs. Correctness is measured separately and not conflated with these timings — eigenvalue agreement across all physical modes ($\leq 0.032\%$) is cross-validated on the identical mesh in `benchmarks/transmon_eigen/results.toml` (Section 7), not in this performance cell. Palace at 16 ranks on the hyperthreaded vCPUs was refused by MPI process binding; we exclude it rather than report a degraded configuration as if it were a measurement.

11 Performance: GPU Cell (Correctness Established; Scaling an Honest Negative)

11.1 Scope

geode-fem’s GPU path today accelerates the *driven* (deterministic frequency-sweep) solve — the matrix-free apply and on-device COCG of Section 3 — and *not* the eigensolve that produced the headline comparison of Section 7, which remains factorization-bound on CPU. A GPU column for the eigenmode benchmark would therefore be misleading; the GPU cell targets the driven pipeline, in two stages: correctness on physical hardware, then a four-configuration scaling study. All GPU execution is CUDA-f32 (Section 3), so accuracy against the f64 reference is reported alongside every timing.

11.2 Correctness on physical hardware

On 2026-07-14, geode-fem code executed on a physical GPU for the first time: an Amazon EC2 `g6e.xlarge` instance (1× NVIDIA L40S, 46 GB; driver 595.71.05, CUDA 13.2). Two CUDA-f32 correctness smokes pass on this hardware: the matrix-free Nédélec matvec smoke (`matrix_free_cuda_f32_smoke`, 15.0s wall including GPU initialization and kernel JIT compilation) and the on-device COCG smoke (`cocg_burn_cuda_f32_smoke`, 3.3s), both asserting agreement with the f64 CPU reference within f32-appropriate tolerances. The quoted wall times are smoke-test durations dominated by startup cost — correctness evidence, not performance measurements. At the time of those smokes the assembled end-to-end `IterativeMatrixFree` path had no GPU runtime test (a disclosed gap, repository issue #499); the scaling study below closes it, running that full path on the GPU across all measured sizes.

11.3 Driven-solve scaling: the measured answer

The scaling cell (repository issue #501; committed as `benchmarks/gpu-driven-scaling/results.toml`) solves one physical problem — a σ -lossy parallel-plate cube with a single lumped port, from the committed driven-solve fixture family — at a fixed frequency across four solver configurations while the mesh is refined from 1,854 to 25,695 edges. All sixteen cells were measured on the *same host* (the `g6e.xlarge` above; 4 vCPUs), so the GPU-versus-CPU comparison is same-host by construction — and, for the same reason, the CPU baselines here are not comparable to the `m6i` numbers of Section 10. This cell is a driven-solve scaling study on a cube fixture family, not the transmon eigenproblem. Table 4 reports the committed medians.

The honest read: **GPU-f32 matrix-free loses to every CPU configuration at every measured size on this host.** The assembled-CSR COCG is $136\times$ faster at 1,854 edges (0.032 versus 4.39s) and still $44\times$ faster at 25,695 edges (1.86 versus 81.8s); even the direct LU is $13\times$ faster at the top size (6.04s). The mechanism is unambiguous: the GPU’s per-iteration cost is kernel-launch dominated at these sizes — ~ 28 ms per iteration at the top size, against ~ 0.63 ms for the assembled-CSR path on CPU — because 25.7k edges is far too small to fill an L40S. Accuracy compounds the verdict: the f32 cells degrade from 1.26×10^{-4} to 1.20×10^{-3} relative error with refinement (the true recomputed residual floors at 6.2×10^{-4} to 5.4×10^{-3} , orders above the requested tolerance), and at the top size convergence is nondeterministic run-to-run — usable for $\sim 0.1\%$ -class field quantities, but trending the wrong way for refinement studies.

One directional positive survives, and it is the one the architecture predicts: against the *same matrix-free algorithm on CPU* (the identical Burn source path on the ndarray backend), the GPU’s deficit falls monotonically from $2.7\times$ slower at 1,854 edges to parity at 25,695 (81.8 versus 82.1s) — the device-resident iteration catches the CPU at ~ 26 k edges and would pass it beyond. That crossover is currently moot in f32, because the top size already sits at the f32 convergence ceiling.

Table 4: Driven-solve wall-clock scaling, solve-only medians of $n = 3$ timed repetitions (warm-up excluded), same host (`g6e.xlarge`: 4 vCPU + 1× L40S). One fixture (σ -lossy lumped-port cube) at fixed ω , mesh-refined across the columns. f64 iterative configurations run at tolerance 10^{-8} ; the f32 GPU configuration at 10^{-6} (its recurrence floor sits above 10^{-8}), so iteration counts are not exactly comparable. The accuracy row is the full-field relative L_2 error of the GPU-f32 solution against the Direct-f64 reference at the same size. At 25,695 edges the GPU-f32 configuration converged in the committed single-frequency runs but its five-point frequency sweep did not, and run-to-run convergence is nondeterministic at that size (f32 floor). Values verbatim from the committed `benchmarks/gpu_driven_scaling/results.toml`.

Configuration	Mesh size (Nédélec edges)			
	1,854	5,859	13,428	25,695
Direct faer LU (CPU f64), s	0.024	0.203	1.540	6.036
Assembled-CSR COCG (CPU f64), s	0.032	0.198	0.709	1.865
Matrix-free COCG (CPU f64), s	1.652	8.885	30.234	82.056
Matrix-free COCG (GPU f32), s	4.388	13.351	34.381	81.764
GPU-f32 accuracy vs. Direct-f64 (rel. L_2)	1.26×10^{-4}	2.93×10^{-4}	4.60×10^{-4}	1.20×10^{-3}

We therefore state the performance position as trajectory, not achievement, with three concrete levers, each tracked: (i) kernel fusion and launch batching to attack the ~ 28 ms per-iteration launch overhead, the dominant cost at benchmark sizes; (ii) f64 (or mixed-precision/iterative-refinement) on device when the cubecf runtime ships it, which removes the convergence ceiling that makes larger problems unreachable — noting honestly that the L40S’s 1/64-rate f64 would trade ceiling for raw rate, and datacenter-class f64 parts change that arithmetic; and (iii) preconditioning the on-device COCG to cut iteration counts, which multiplies whatever the first two levers yield. Nothing in Table 4 contradicts the device-resident-Krylov design intent; it bounds where that design pays off — well above 26k edges, in f64 — and we prefer publishing the bound to implying the win.

12 Reproducibility

Every artifact behind every number in this paper is committed and scripted.⁴

- **Geometry and mesh.** Generated by DeviceLayout.jl v1.15.0’s `SingleTransmon` example ([AWS Center for Quantum Computing, 2025](#)); the fixture generator is committed, and the MSH 4.1 fixture is pinned by SHA-256 (5b3ff4c3...b33dd; full hash and provenance in the committed `transmon_smoke.provenance.txt`).
- **The MSH conversion gotcha.** MFEM’s MSH 4.1 reader rejects the DeviceLayout multi-entity-block file (“vertices indices are not unique”), so the Palace runs consume an MSH 2.2 conversion (`gmsh -save -format msh2`). The conversion is node-preserving (all 22,684 nodes unchanged, physical groups preserved) and physics-neutral: eigenvalues reproduce bit-for-bit across the conversion. The conversion step is documented in the committed `palace_config.provenance.txt`. We document this because it is exactly the kind of silent pipeline step a same-mesh claim must disclose.
- **Solver configurations.** The Palace configuration and its provenance file are committed under `reference/fixtures/transmon_palace/`; geode-fem’s benchmark definitions live in `benchmarks/transmon_eigen/results.toml`, alongside the exact benchmark commands.

⁴Repository URL and archival DOI for the geode-fem source at commit 3174015 to be added at submission. TODO(operator).

- **Precision of the committed agreement numbers.** `benchmarks/transmon_eigen/results.toml` stores geode-fem frequencies at three decimals while the Palace column carries full precision, so the committed per-mode `rel_err_pct` is computed against the rounded geode-fem values. At full precision the agreement is slightly better (approximately 0.029% worst case). We quote the committed-artifact numbers throughout and disclose the rounding here.
- **Performance artifacts.** The complete CPU cell (per-run wall clocks, peak RSS, instance and software provenance) is committed as `benchmarks/transmon_bench_cpu/results.toml`. The complete GPU driven-solve scaling cell (all sixteen configuration $\times$ size cells, iteration counts, recomputed true residuals, accuracy-versus-Direct columns, DNF/flaky handling, and same-host provenance) is committed as `benchmarks/gpu_driven_scaling/results.toml`.
- **Tripwire, gauge-arc, and spurious-mode artifacts.** The measured L-doubling tripwire values (17.4901 $\rightarrow$ 12.37 GHz, ratio 0.7071), the spurious-mode measurements (3.4528 GHz, $p = 0.9942$; L-doubling 3.4528 $\rightarrow$ 2.4449 GHz, ratio 0.7081), and the full gauge/projection arc of Section 9 — tree-cotree counts and the 1.64% shift, the bulk-projection cavity/junction measurements (divergence ratio 50.2; projected norm 1.06×10^{-4}), and the port-aware acceptance (all six modes, worst case 0.029%) — are committed in `benchmarks/transmon_eigen/results.toml`, each pinned by a release-tier regression test.
- **Oracles.** Palace’s raw outputs (`eig.csv`, per-port participation, run log) are committed; a rerun from the committed configuration reproduced `eig.csv` bit-for-bit.
- **Hardware disclosure.** CPU cell: EC2 `m6i.4xlarge` (8 physical cores / 16 vCPU, 64 GB), `us-west-2`. GPU cell (correctness and scaling): EC2 `g6e.xlarge` (4 vCPU, 30 GB; $1 \times$ NVIDIA L40S 46 GB; driver 595.71.05, CUDA 13.2).
- **Versions.** geode-fem commit 3174015 (headline cells; post-measurement merges are cited by PR number where discussed); Palace commit `fba6a5b`; DeviceLayout.jl v1.15.0; Burn/burn-cuda 0.21.

13 Discussion

What the cross-validation does and does not establish. In V&V terms (Oberkamp and Roy, 2010; Roache, 1997), this study is code verification and code-to-code comparison, not validation against nature: two independent implementations agreeing to $\leq 0.033\%$ (worst case 0.032%) on the identical discrete problem, with analytic tripwires pinning the junction physics. It does not certify that either solver’s answer matches a measured device — that is what Ye et al. (2025)-style experimental benchmarks address, and the two evidence types are complementary. What same-mesh agreement uniquely rules out is the large class of errors that live in formulation, assembly, boundary-condition handling, and eigensolver machinery — and for the architecture thesis specifically, it rules out the possibility that phrasing assembly and operators as batched ML-stack tensor algebra silently costs accuracy on a production geometry.

What the performance evidence supports. The measured position is: per-core CPU competitiveness demonstrated (a single geode-fem core at 28.7s beats the distributed reference on eight ranks at 44.5s at the 133k-DOF physical target, at $\sim 12\times$ fewer core-seconds, with the gap widening off-target and inverting only at $\sim 1.16\text{M}$ DOFs where the direct factorization is memory-bound), GPU portability demonstrated at essentially zero incremental engineering (the CUDA leg is the same

source path as the CPU leg, a backend type parameter away), and GPU *advantage* not demonstrated — the scaling cell of Section 11 is an honest negative at benchmark sizes, with a measured parity crossover against the same algorithm on CPU at $\sim 26k$ edges and named, tracked levers beyond it. The claim this paper stands behind is therefore viability-plus-trajectory, not achieved GPU speedup.

Threats to validity. A common-mode error — both solvers wrong in the same way — cannot be excluded by cross-comparison alone; the analytic anchors (f_{LC} , the $1/\sqrt{2}$ scaling) and the genuinely disjoint implementations make it unlikely for the junction physics specifically. Configuration-entry mismatch (comparing subtly different problems) is mitigated by committed configurations, the shared mesh file, and participation-based mode identification on the geode-fem side with frequency-anchored cross-solver assignment (Section 6). Performance numbers are $n = 3$ measurements on one instance type per cell; they support the architecture-level readings in Sections 10–11, not microsecond-level claims, and the GPU cell’s CPU baselines are same-host (4-vCPU) values that must not be compared across boxes.

Toward qubit quantities. A first quantum layer above this benchmark has since been built and measured on the same mesh (committed as `benchmarks/transmon_quantum/results.toml`): a tensor- ϵ electrostatic solve on the mesh’s three node-disjoint metal conductors yields the island Maxwell capacitance $C_\Sigma = 136.7$ fF, hence $E_C = 0.142$ GHz and $E_J/E_C = 77.6$, and the charge-basis-exact transmon spectrum of Koch et al. (2007) gives $\omega_{01} = 3.38$ GHz and anharmonicity $\alpha = -0.158$ GHz — a plausible transmon, computed end-to-end from the same committed geometry. It carries its own honest finding: C_Σ exceeds the ~ 90 fF design-anchor value, and the discrepancy is diagnosed as a model-definition difference rather than a truncation artifact (grounding the exterior wall moves C_Σ by a relative 6×10^{-5} ; the scalar-versus-tensor sapphire treatment shifts it 0.75%). Connecting this electrostatic route to the field-mode EPR quantization (Mineev et al., 2021a; Nigg et al., 2012) over the eigenmodes of Section 7 is a separate paper’s worth of analysis and is deliberately not compressed into this one.

Limitations. (i) One geometry, one mesh, first order only — higher-order and mesh-refinement convergence is future work (a $p = 2$ Palace configuration is already committed for that purpose). (ii) The lossless v1 model leaves readout ports open; resistive terminations (and hence complex-symmetric pencils and mode Q s) are out of scope. (iii) The headline eigenpath remains the ungauged committed benchmark; the port-aware projection of Section 9 retains all six modes at $\leq 0.029\%$ but is a composite solver (an ungauged extract feeds it), and the solenoidal port artifact is characterized and filtered rather than eliminated. (iv) The per-core efficiency and single-core-beats-eight-ranks result is specific to the small-to-medium regime where a serial direct factorization is memory-feasible; it is explicitly scale-bounded and inverts at $\sim 1.16M$ DOFs, where the direct path is OOM-killed and the distributed iterative reference completes. (v) The GPU evidence covers driven-solve correctness and an honest scaling negative on a cube fixture family well below GPU-saturating sizes, all in f32; there is no GPU eigensolve path, and no GPU claim in this paper extends beyond what Table 4 contains.

14 Conclusion

We set out to test whether a general-purpose ML tensor-compiler stack can carry production-grade computational electromagnetics, and to hold the answer to a cross-validation standard rather than a demonstration standard. The evidence: a Burn-native full-wave $H(\text{curl})$ solver whose assembly is batched tensor algebra reproduces Palace — the domain-specific reference stack, itself built

on a FEM-community element JIT — to $\leq 0.033\%$ across all six eigenmodes of a real transmon geometry on the identical mesh (worst case 0.032% ; 0.001% on the junction LC mode), with exact analytic tripwires guarding the junction physics; it beats the reference on per-core efficiency (one core at 28.7s versus eight ranks at 44.5s at the 133k-DOF target, $\sim 12\times$ fewer core-seconds, a target-robust direct-versus-iterative advantage that is scale-bounded at $\sim 1.16\text{M}$ DOFs); and it gains a portable CUDA execution path from the same source, whose measured scaling cell we report as the honest negative it currently is — kernel-launch-bound losses at every benchmark size, a parity crossover against its own algorithm on CPU at $\sim 26\text{k}$ edges, and named levers (fusion, on-device f64, preconditioning) between here and advantage. Along the way the benchmark delivered the first independent validation of the Palace/DeviceLayout.jl transmon stack, a reusable oracle-first cross-solver methodology, and an honest-physics record upgraded from disclosure to diagnosis: the by-construction absent qubit mode confirmed by both solvers, and the spurious mode run to ground through a measured gauge/projection arc that ends with all six physical modes retained at $\leq 0.029\%$ and the artifact characterized as a port-surface resonance. The architectural conclusion is measured but real: the ML ecosystem’s compiler investment is inheritable by finite-element electromagnetics at production accuracy today, with the performance frontier — not the correctness frontier — as the open work.

Acknowledgments. geode-fem’s development was AI-orchestrated (agent-driven implementation with human direction and review); this benchmark program and paper follow the same discipline, with every reported number traced to a committed, human-auditable artifact.

References

- R. Albanese and G. Rubinacci. Solution of three dimensional eddy current problems by integral and differential methods. *IEEE Transactions on Magnetics*, 24(1):98–101, 1988. doi: 10.1109/20.43865. URL <https://doi.org/10.1109/20.43865>.
- Martin S. Alnæs, Anders Logg, Kristian B. Ølgaard, Marie E. Rognes, and Garth N. Wells. Unified form language. *ACM Transactions on Mathematical Software*, 40(2):1–37, 2014. doi: 10.1145/2566630. URL <https://doi.org/10.1145/2566630>.
- Robert Anderson, Julian Andrej, Andrew Barker, Jamie Bramwell, Jean-Sylvain Camier, Jakub Cervený, Veselin Dobrev, Yohann Dudouit, Aaron Fisher, Tzanio Kolev, Will Pazner, Mark Stowell, Vladimir Tomov, Ido Akkerman, Johann Dahm, David Medina, and Stefano Zampini. MFEM: a modular finite element methods library. *Computers & Mathematics with Applications*, 81:42–74, 2021. doi: 10.1016/j.camwa.2020.06.009.
- Julian Andrej, Nabil Atallah, Jan-Phillip Bäcker, Jean-Sylvain Camier, Dylan Copeland, Veselin Dobrev, Yohann Dudouit, Tobias Duswald, Brendan Keith, Dohyun Kim, Tzanio Kolev, Boyan Lazarov, Ketan Mittal, Will Pazner, Socratis Petrides, Syun’ichi Shiraiwa, Mark Stowell, and Vladimir Tomov. High-performance finite elements with mfem. *The International Journal of High Performance Computing Applications*, 2024. doi: 10.1177/10943420241261981. URL <https://doi.org/10.1177/10943420241261981>.
- Alexander Arsenovic, Julien Hillairet, Jackson Anderson, Henrik Forsten, Vincent Ries, Michael Eller, Noah Sauber, Robert Weikle, William Barnhart, and Franz Forstmayr. scikit-rf: An open source python package for microwave network creation, analysis, and calibration [speaker’s corner]. *IEEE Microwave Magazine*, 23(1):98–105, 2022. doi: 10.1109/mmm.2021.3117139. URL <https://doi.org/10.1109/mmm.2021.3117139>.

- AWS Center for Quantum Computing. DeviceLayout.jl: computer-aided design of quantum integrated circuits. <https://github.com/aws-cqc/DeviceLayout.jl>, 2025. v1.15.0 used for geometry/mesh generation in this work. No journal or arXiv artifact exists as of 2026-07-14 (litsearch absence check).
- AWS Quantum Technologies Blog. AWS releases open-source software Palace for cloud-based electromagnetics simulations of quantum computing hardware. <https://aws.amazon.com/blogs/quantum-computing/aws-releases-open-source-software-palace-for-cloud-based-electromagnetics-simulations-of-quantum-computing-hardware/>, 2023.
- Jed Brown, Ahmad Abdelfattah, Valeria Barra, Natalie Beams, Jean-Sylvain Camier, Veselin Dobrev, Yohann Dudouit, Leila Ghaffari, Tzanio Kolev, David Medina, Will Pazner, Thilina Ratnayaka, Jeremy Thompson, and Stan Tomov. libceed: Fast algebra for high-order element-based discretizations. *Journal of Open Source Software*, 6(63):2945, 2021. doi: 10.21105/joss.02945. URL <https://doi.org/10.21105/joss.02945>.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: An automated end-to-end optimizing compiler for deep learning, 2018. URL <https://arxiv.org/abs/1802.04799>.
- Mingyuan Chi and Shizheng Wen. torch-sla: Differentiable sparse linear algebra with adjoint solvers and sparse tensor parallelism for pytorch, 2026. URL <https://arxiv.org/abs/2601.13994>.
- P. Dular, C. Geuzaine, F. Henrotte, and W. Legros. A general environment for the treatment of discrete problems and its application to the finite element method. *IEEE Transactions on Magnetics*, 34(5):3395–3398, 1998. doi: 10.1109/20.717799. URL <https://doi.org/10.1109/20.717799>.
- Christophe Geuzaine and Jean-François Remacle. Gmsh: a 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009. doi: 10.1002/nme.2579.
- Alec M. Hammond, Ardavan Oskooi, Mo Chen, Zin Lin, Steven G. Johnson, and Stephen E. Ralph. High-performance hybrid time/frequency-domain topology optimization for large-scale photonics inverse design. *Optics Express*, 30(3):4467, 2022. doi: 10.1364/oe.442074. URL <https://doi.org/10.1364/oe.442074>.
- Vicente Hernandez, Jose E. Roman, and Vicente Vidal. SLEPc: a scalable and flexible toolkit for the solution of eigenvalue problems. *ACM Transactions on Mathematical Software*, 31(3):351–362, 2005. doi: 10.1145/1089014.1089019.
- Ralf Hiptmair and Jinchao Xu. Nodal auxiliary space preconditioning in $H(\text{curl})$ and $H(\text{div})$ spaces. *SIAM Journal on Numerical Analysis*, 45(6):2483–2509, 2007. doi: 10.1137/060660588.
- Johannes Hoffmann, Christian Hafner, Patrick Leidenberger, Jan Hesselbarth, and Sven Burger. Comparison of electromagnetic field solvers for the 3d analysis of plasmonic nano antennas, 2009. URL <https://arxiv.org/abs/0907.3570>.
- Philipp Holl, Vladlen Koltun, and Nils Thuerey. Learning to control pdes with differentiable physics, 2020. URL <https://arxiv.org/abs/2001.07457>.

- Miklós Homolya, Lawrence Mitchell, Fabio Luporini, and David A. Ham. Tsfc: A structure-preserving form compiler. *SIAM Journal on Scientific Computing*, 40(3):C401–C428, 2018. doi: 10.1137/17m1130642. URL <https://doi.org/10.1137/17m1130642>.
- Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Frédo Durand. DiffTaichi: Differentiable programming for physical simulation, 2019. URL <https://arxiv.org/abs/1910.00935>.
- Tyler W. Hughes, Ian A. D. Williamson, Momchil Minkov, and Shanhui Fan. Forward-mode differentiation of maxwell’s equations. *ACS Photonics*, 6(11):3010–3016, 2019. doi: 10.1021/acsphotonics.9b01238. URL <https://doi.org/10.1021/acsphotonics.9b01238>.
- Jiantao Jiang, Yanli Wang, Yifan Wang, and Hehu Xie. Fieldtnn-based machine learning method for maxwell eigenvalue problems, 2024. URL <https://arxiv.org/abs/2411.15828>.
- Sarah Quiñones El Kazdadi. faer: A linear algebra library for the rust programming language. *Journal of Open Source Software*, 11(123):6099, 2026. doi: 10.21105/joss.06099. URL <https://doi.org/10.21105/joss.06099>.
- Yongha Kim, Anthony W. Jung, Sanmun Kim, Kevin Octavian, Doyoung Heo, Chaejin Park, Jeongmin Shin, Sunghyun Nam, Chanhung Park, Juho Park, Sangjun Han, Jinmyoung Lee, Seolho Kim, Min Seok Jang, and Chan Y. Park. Meent: Differentiable electromagnetic simulator for machine learning, 2024. URL <https://arxiv.org/abs/2406.12904>.
- Robert C. Kirby and Anders Logg. A compiler for variational forms. *ACM Transactions on Mathematical Software*, 32(3):417–444, 2006. doi: 10.1145/1163641.1163644. URL <https://doi.org/10.1145/1163641.1163644>.
- Jens Koch, Terri M. Yu, Jay Gambetta, A. A. Houck, D. I. Schuster, J. Majer, Alexandre Blais, M. H. Devoret, S. M. Girvin, and R. J. Schoelkopf. Charge-insensitive qubit design derived from the Cooper pair box. *Physical Review A*, 76:042319, 2007. doi: 10.1103/PhysRevA.76.042319.
- Tzanio V. Kolev and Panayot S. Vassilevski. Parallel auxiliary space AMG for $H(\text{curl})$ problems. *Journal of Computational Mathematics*, 27(5):604–623, 2009. doi: 10.4208/jcm.2009.27.5.013. URL <https://doi.org/10.4208/jcm.2009.27.5.013>. Crossref records the online-publication year 2018; print year 2009 used here (litsearch cosmetic fix, see version-dir refs.bib header).
- Richard B. Lehoucq, Danny C. Sorensen, and Chao Yang. *ARPACK Users’ Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. SIAM, 1998. doi: 10.1137/1.9780898719628.
- Liang Liang, Minliang Liu, John Eleftheriades, and Wei Sun. Pytorch-fea: Autograd-enabled finite element analysis methods with applications for biomechanical analysis of human aorta. *Computer Methods and Programs in Biomedicine*, 238:107616, 2023. doi: 10.1016/j.cmpb.2023.107616. URL <https://doi.org/10.1016/j.cmpb.2023.107616>.
- Anders Logg and Garth N. Wells. Dolfin: Automated finite element computing. *ACM Transactions on Mathematical Software*, 37(2):1–28, 2010. doi: 10.1145/1731022.1731030. URL <https://doi.org/10.1145/1731022.1731030>.
- Yannik Mahlau, Frederik Schubert, Lukas Berg, and Bodo Rosenhahn. Fdtdx: High-performance open-source fdtd simulation with automatic differentiation. *Journal of Open Source Software*, 11(117):8912, 2026. doi: 10.21105/joss.08912. URL <https://doi.org/10.21105/joss.08912>.

- J.B. Manges and Z.J. Cendes. A generalized tree-cotree gauge for magnetic field computation. *IEEE Transactions on Magnetics*, 31(3):1342–1347, 1995. doi: 10.1109/20.376275. URL <https://doi.org/10.1109/20.376275>.
- Huan-Ting Meng, Bao-Lin Nie, Steven Wong, Charles Macon, and Jian-Ming Jin. Gpu accelerated finite-element computation for electromagnetic analysis. *IEEE Antennas and Propagation Magazine*, 56(2):39–62, 2014. doi: 10.1109/map.2014.6837065. URL <https://doi.org/10.1109/map.2014.6837065>.
- Zlatko K. Minev, Zaki Leghtas, Shantanu O. Mundhada, Lysander Christakis, Ioan M. Pop, and Michel H. Devoret. Energy-participation quantization of Josephson circuits. *npj Quantum Information*, 7:131, 2021a. doi: 10.1038/s41534-021-00461-8.
- Zlatko K. Minev, Thomas G. McConkey, Maika Takita, Antonio D. Corcoles, and Jay M. Gambetta. Circuit quantum electrodynamics (cqed) with modular quasi-lumped models, 2021b. URL <https://arxiv.org/abs/2103.10344>.
- Jean-Claude Nédélec. Mixed finite elements in $\mathbb{R}^3$. *Numerische Mathematik*, 35(3):315–341, 1980. doi: 10.1007/BF01396415.
- Simon E. Nigg, Hanhee Paik, Brian Vlastakis, Gerhard Kirchmair, S. Shankar, Luigi Frunzio, M. H. Devoret, R. J. Schoelkopf, and S. M. Girvin. Black-box superconducting circuit quantization. *Physical Review Letters*, 108:240502, 2012. doi: 10.1103/PhysRevLett.108.240502.
- William L. Oberkampf and Christopher J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010. doi: 10.1017/cbo9780511760396. URL <https://doi.org/10.1017/cbo9780511760396>.
- Palace developers, AWS Center for Quantum Computing. Palace: PArallel LARge-scale Computational Electromagnetics. <https://github.com/aws-labs/palace>, 2023. Open-source 3D finite-element solver; v0.13-class, git fba6a5b used in this work. No associated journal article exists as of 2026-07 (absence re-confirmed by litsearch, 2026-07-14).
- Greg Peairs. Schematic-driven design of a quantum processor with DeviceLayout.jl. JuliaCon Global 2025, <https://pretalx.com/juliacon-2025/talk/7EYFPN/>, 2025. Recording: <https://www.youtube.com/watch?v=-LohBVtFdwU>.
- Greg Peairs and Hugh Carson. Design quantum integrated circuits with open-source software DeviceLayout.jl from AWS. <https://aws.amazon.com/blogs/quantum-computing/design-quantum-integrated-circuits-with-open-source-software-devicelayout-jl-from-aws/>, 2025. AWS Quantum Technologies Blog; source of the transmon+resonator geometry and the reference workflow reproduced here.
- Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide. *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 519–530, 2013. doi: 10.1145/2491956.2462176. URL <https://doi.org/10.1145/2491956.2462176>.
- Florian Rathgeber, David A. Ham, Lawrence Mitchell, Michael Lange, Fabio Luporini, Andrew T. T. Mcrae, Gheorghe-Teodor Bercia, Graham R. Markall, and Paul H. J. Kelly. Firedrake. *ACM Transactions on Mathematical Software*, 43(3):1–27, 2017. doi: 10.1145/2998441. URL <https://doi.org/10.1145/2998441>.

- P. J. Roache. Quantification of uncertainty in computational fluid dynamics. *Annual Review of Fluid Mechanics*, 29(1):123–160, 1997. doi: 10.1146/annurev.fluid.29.1.123. URL <https://doi.org/10.1146/annurev.fluid.29.1.123>.
- Martin F. Schubert, Alfred K. C. Cheung, Ian A. D. Williamson, Aleksandra Spyra, and David H. Alexander. Inverse design of photonic devices with strict foundry fabrication constraints, 2022. URL <https://arxiv.org/abs/2201.12965>.
- Sadman Shanto, Andre Kuo, Clark Miyamoto, Haimeng Zhang, Vivek Maurya, Evangelos Vlachos, Malida Hecht, Chung Wa Shum, and Eli Levenson-Falk. Squadds: A validated design database and simulation workflow for superconducting qubit design. *Quantum*, 8:1465, 2024. doi: 10.22331/q-2024-09-09-1465. URL <https://doi.org/10.22331/q-2024-09-09-1465>.
- David Sommers, Zach Degnan, Divita Gautam, Yi-Hsun Chen, Chun-Ching Chiu, Arkady Fedorov, and Prasanna Pakkiam. Open-source highly parallel electromagnetic simulations for superconducting circuits, 2025. URL <https://arxiv.org/abs/2511.01220>.
- Tracel AI. Burn: a comprehensive deep-learning and tensor-computation framework in Rust. <https://github.com/tracel-ai/burn>, 2024. v0.21.0 used. No journal/arXiv paper exists (litsearch check 2026-07-14): the repo CITATION.cff lists authors Simard, Fortier-Dubois, Tadjibaev, Lagrange + Burn Framework Contributors, with no DOI and no preferred-citation paper — cited as software.
- Shizheng Wen, Mingyuan Chi, Tianwei Yu, Ben Moseley, Mike Yan Michelis, Pu Ren, Hao Sun, and Siddhartha Mishra. Learning, solving and optimizing pdes with tensorgalerkin: an efficient high-performance galerkin assembly algorithm, 2026. URL <https://arxiv.org/abs/2602.05052>.
- Tianju Xue, Shuheng Liao, Zhengtao Gan, Chanwook Park, Xiaoyu Xie, Wing Kam Liu, and Jian Cao. Jax-fem: A differentiable gpu-accelerated 3d finite element solver for automatic inverse design and mechanistic data science. *Computer Physics Communications*, 291:108802, 2023. doi: 10.1016/j.cpc.2023.108802. URL <https://doi.org/10.1016/j.cpc.2023.108802>.
- Jiale Ye, Jiaheng Wang, and Yu-xi Liu. Electromagnetic feature extraction in superconducting quantum circuits: An open-source finite-element workflow using palace, 2025. URL <https://arxiv.org/abs/2511.09041>.